OpenCesium

A Meta-Agent Workflow Automation Environment

for Real-World Tool-Using Agent Workflow Automations

Abstract. OpenCesium is an OpenEnv-compliant benchmark environment in which a meta-agent learns to construct, parameterize, and iteratively refine executable tool-using sub-agents so as to complete structured business workflow tasks. The environment is grounded in real-world automation scenarios — financial data retrieval, economic computation, and multi-source report generation — and provides a shaped, trajectory-dense reward signal over a sequential decision process. Three tasks of escalating complexity serve as the task suite, each equipped with a deterministic, composable grader. The full system is designed for deployment on Hugging Face Spaces via Docker and satisfies all mandatory OpenEnv v1.0 requirements.

Specification Version	1.0
Environment ID	`opencesium-v1`
OpenEnv Compliance	Full (v1.0)
API Endpoints	`reset()` / `step()` / `state()`
Task Count	3 — Easy, Medium, Hard
Reward Type	Dense, shaped, trajectory-level
Deployment Target	Hugging Face Spaces + Docker
Inference Constraint	<20 min · vCPU = 2 · RAM = 8 GB
LLM Client	OpenAI API (via `API_BASE_URL`)

Contents

1 Introduction and Motivation **3**
1.1 Research Context . . . 3
1.2 Core Thesis . . . 3
1.3 Why Workflow Automation? . . . 3
1.4 Document Structure . . . 3

2 Formal Model **3**
2.1 Entity Definitions . . . 3
2.2 Sequential Decision Process . . . 4
2.3 Performance Functional . . . 4
2.4 Termination Conditions . . . 4
2.5 Action Validation Layer . . . 4

3 OpenEnv v1.0 Specification Compliance **4**
3.1 Typed Data Models . . . 5
3.2 Observability . . . 5
3.3 API Contract . . . 5
3.4 State Isolation . . . 6
3.5 Dual Execution Mode . . . 6
3.6 Compliance Summary . . . 6
3.7 Safety Guarantees . . . 6

4 Task Suite **7**
4.1 Task Design Principles . . . 7
4.2 Task 1 — Financial Alert Report (Easy) . . . 7
4.3 Task 2 — GDP Growth Analysis (Medium) . . . 7
4.4 Task 3 — Multi-Source Structured Report (Hard) . . . 8
4.5 Structural Validation . . . 8

5 Reward Function **8**
5.1 Design Rationale . . . 8
5.2 Reward Decomposition . . . 8
5.3 Reward Computation Pipeline . . . 10
5.4 Illustrative Step-Level Rewards . . . 10
5.5 Cumulative Episode Reward . . . 10

6 Tool Abstraction Layer **10**
6.1 Tool Interface . . . 10
6.2 Tool Registry . . . 11
6.3 External API Binding . . . 11
6.4 Cost Model . . . 11
6.5 Development vs. Production Mode . . . 11

7 Implementation Plan **11**
7.1 Repository Structure . . . 11
7.2 Development Phases . . . 12

7.3 Testing Strategy 12
7.4 Strategy Space 13

8 Baseline Inference System **13**
8.1 Constraints 13
8.2 Inference Script 13
8.3 Expected Baseline Scores 14

9 Deployment Architecture **14**
9.1 Dockerfile 14
9.2 openenv.yaml 15

10 Evaluation and Judging Alignment **15**
10.1 Judging Criteria Mapping 16
10.2 Pre-Submission Checklist 16
10.3 Failure Mode Mitigations 17
10.4 Novel Contributions Summary 17

1. Introduction and Motivation

1.1. Research Context

Modern AI agent research has converged on a dominant paradigm: a single language model acting as a policy that issues tool calls, synthesises observations, and iterates toward task completion. Benchmarks such as WebArena [1], ToolBench [2], and AgentBench [3] measure this single-agent execution competence in depth. However, a systematically under-explored problem exists one level above: *how well can an AI agent reason about the construction of another agent?*

This meta-level question is not merely academic. Production deployments of workflow automation systems (AutoGPT, LangChain pipelines, n8n, Zapier AI) require a human or automated orchestrator to decide which tools to compose, in what order to invoke them, and how to configure their prompts and constraints. Getting this design decision wrong results in broken pipelines, wasted compute, and incorrect outputs. There is currently no rigorous RL environment that benchmarks this capability.

OpenCesium fills this gap.

1.2. Core Thesis

Problem Statement

OpenCesium formalises agent construction as a first-class sequential decision problem. A meta-agent interacts with the environment by issuing configuration actions that build, modify, and evaluate a tool-using worker agent. The environment provides partial reward at every step, enabling learning signals over the full construction trajectory rather than only at execution outcome.

1.3. Why Workflow Automation?

Workflow automation satisfies the hackathon's requirement that the environment simulate a task *humans actually perform*. Enterprises routinely maintain pipelines that: (i) pull live financial or economic data from external APIs; (ii) perform structured numerical computation over that data; (iii) assemble formatted analytical artifacts; and (iv) deliver those artifacts through communication channels. Designing such pipelines — selecting the right tools, sequencing them correctly, and wiring their inputs and outputs — is cognitively demanding and practically high-value. It therefore constitutes an ideal benchmark domain for meta-agent evaluation.

1.4. Document Structure

This document proceeds as follows. Section 2 presents the formal mathematical model. Section 3 details OpenEnv v1.0 compliance. Section 4 specifies the three-task suite. Section 5 develops the reward function. Section 6 describes the tool abstraction layer. Section 7 covers the implementation plan. Section 8 specifies the baseline inference system. Section 9 covers deployment. Section 10 addresses evaluation and judging alignment.

2. Formal Model

2.1. Entity Definitions

Definition 1 (Tool)**.** *A tool $u \in \mathcal{U}$ is a deterministic callable mapping $u : \mathcal{X}_u \to \mathcal{Y}_u$, where $\mathcal{X}_u$ and $\mathcal{Y}_u$ are its typed input and output domains respectively.*

Definition 2 (Worker Agent)**.** *A worker agent is a triple $A = (\pi_a, \mathcal{U}_A, \theta)$ where π_a is the agent's*

LLM policy, $\mathcal{U}_A \subseteq \mathcal{U}$ is its assigned tool subset, and $\theta = (\text{prompt}, k_{\max}, \text{strategy})$ is its configuration vector.

Definition 3 (Task)**.** *A task is $T = (G, \mathcal{U}_T, \Phi_T, \kappa)$ where G is a natural language goal, $\mathcal{U}_T \subseteq \mathcal{U}$ is the set of tools required for full credit, $\Phi_T : (A, G) \to [0, 1]$ is the grading function, and κ is the step budget.*

Definition 4 (Meta-Action)**.** *A meta-action $a_m \in \mathcal{A}_m$ is drawn from:*

$$\mathcal{A}_m = \{\textsc{AddTool},\ \textsc{RemoveTool},\ \textsc{SetPrompt},\ \textsc{SetStrategy},\ \textsc{Evaluate},\ \textsc{Noop}\}$$

2.2. Sequential Decision Process

OpenCesium is modelled as a finite-horizon **Markov Decision Process** $\mathcal{M} = (\mathcal{S}, \mathcal{A}_m, \mathcal{T}, R, \gamma, H)$:

$$s_t = \big(T,\ A_t,\ h_t,\ r_{t-1}\big) \in \mathcal{S} \qquad s_{t+1} = \mathcal{T}(s_t,\ a_t)$$

$$R_t = \underbrace{\Phi_T(A_{t+1}, T) - \Phi_T(A_t, T)}_{\text{performance gain}} - \underbrace{\lambda\, C(a_t)}_{\text{action cost}}$$

$$\max_{\pi_m}\ \mathbb{E}_{\pi_m}\left[\sum_{t=0}^{H} \gamma^t R_t\right]$$

where $h_t = \{(a_0, r_0), \ldots, (a_{t-1}, r_{t-1})\}$ is the interaction history, $C : \mathcal{A}_m \to \mathbb{R}_{\geq 0}$ is an action cost function, $\lambda \geq 0$ is a regularisation coefficient, and $H = \kappa$ is the episode horizon.

2.3. Performance Functional

The grading functional Φ_T decomposes into interpretable sub-criteria:

$$\Phi_T(A, T) = \sum_{i=1}^{N} w_i \cdot f_i(A, T), \qquad \sum_i w_i = 1, \quad f_i \in [0, 1]$$

The sub-criteria f_i include: *output correctness*, *required tool coverage*, *step efficiency*, and *structural completeness* (for the hard task).

2.4. Termination Conditions

Episode termination τ satisfies:

$$\tau = \min\big\{t \mid t \geq H \ \text{ or } \ \big(\textsc{Evaluate} \in a_t \ \text{ and } \ \Phi_T(A_t, T) \geq \theta_{\text{pass}}\big)\big\}$$

2.5. Action Validation Layer

Before execution, each action is validated:

- Unknown `action_type` → converted to `NOOP` with penalty
- Invalid payload schema → logged in `error_log`
- Tool not in registry → ignored with penalty

This ensures robustness against malformed LLM outputs and prevents runtime failures.

3. OpenEnv v1.0 Specification Compliance

3.1. Typed Data Models

All interface types are defined as Pydantic v2 models, ensuring schema enforcement, JSON serialisation, and `openenv validate` compatibility.

Listing 1: Core Pydantic Models

```python
from pydantic import BaseModel, Field
from typing import List, Literal, Dict, Any, Optional

class Observation(BaseModel):
    task_id:              str
    task_description:     str
    available_tools:      List[str]
    agent_config:         Dict[str, Any]
    last_score:           float = 0.0
    last_reward:          float = 0.0
    step_index:           int   = 0
    max_steps:            int
    history:              List[Dict[str, Any]] = Field(default_factory=list)
    tool_execution_log:   List[str]            = Field(default_factory=list)
    error_log:            List[str]            = Field(default_factory=list)
    done:                 bool  = False

class Action(BaseModel):
    action_type: Literal[
        "ADD_TOOL","REMOVE_TOOL","SET_PROMPT",
        "SET_STRATEGY","EVALUATE","NOOP"
    ]
    payload: Dict[str, Any] = Field(default_factory=dict)

class Reward(BaseModel):
    total:      float
    components: Dict[str, float]
```

3.2. Observability

The environment maintains:

- `tool_execution_log`: ordered sequence of tool invocations
- `error_log`: invalid actions, failures, parsing errors

These logs are used for:

- reward computation (loop penalties)
- grader validation
- debugging and reproducibility

3.3. API Contract

Listing 2: OpenEnv API — Public Interface

```python
class OpenCesiumEnv:
    def reset(self) -> Observation:
        """Return clean initial observation. Zero step count, empty config."""
        ...

    def step(self, action: Action) -> tuple[Observation, Reward, bool, dict]:
        """
        Apply action. Returns:
          obs        updated Observation
          reward     Reward with component breakdown
          done       termination flag
          info       auxiliary diagnostics dict
        """
```

```
        ...

    def state(self) -> dict:
        """Return serialisable snapshot of current EnvState."""
        ...
```

3.4. State Isolation

Each episode is strictly isolated:

- no cross-episode memory
- agent configuration reset on `reset()`
- logs cleared between episodes

Ensures reproducibility and fair evaluation.

3.5. Dual Execution Mode

OpenCesium supports two execution pathways:

- **Direct Python Mode (Inference Runtime)** The baseline inference script interacts directly with the environment:

```
env = OpenCesiumEnv(...)
obs = env.reset()
obs, reward, done, info = env.step(action)
```

- **HTTP Mode (Validator Interface)** For OpenEnv validation and Hugging Face deployment:
  - `POST /reset`
  - `POST /step`
  - `POST /state`

Both modes share identical transition logic and EnvState.

3.6. Compliance Summary

Table 1: OpenEnv v1.0 Specification Compliance Matrix

Requirement	Implementation	Status
Typed Observation model	Pydantic v2, 11 fields, JSON-serialisable	✓
Typed Action model	Pydantic v2, 6 action types via `Literal`	✓
Typed Reward model	`total: float` + component dict	✓
`reset()` clean init	Zero step, empty config, full obs returned	✓
`step()` returns tuple	(obs, reward, done, info) confirmed	✓
`state()` serialisable	Returns `dict` via `EnvState.model_dump()`	✓
`openenv.yaml` present	Metadata, spaces, tasks, runtime documented	✓
≥ 3 tasks with graders	Easy, Medium, Hard; graders return $[0, 1]$	✓
Graders deterministic	No RNG; pure function of output	✓
Dense reward	Partial credit at every step	✓
Real-world domain	Workflow automation (finance + email)	✓

3.7. Safety Guarantees

- Step limit enforces termination
- Tool calls are time-bounded
- No external side effects in development mode

- Invalid actions safely handled

4. Task Suite

4.1. Task Design Principles

Each task is constructed to satisfy four properties simultaneously: (i) *grounded* — the goal mirrors a genuine business operation; (ii) *compositional* — correct execution requires multiple distinct tools in a specific dependency order; (iii) *gradable* — success can be measured deterministically without a judge model; and (iv) *differentially difficult* — the task set spans a wide capability range from basic two-tool pipelines to multi-source DAG workflows.

4.2. Task 1 — Financial Alert Report (Easy)

`task_easy_stock_alert`

Goal. Retrieve the real-time price of AAPL from a financial data API, format a concise one-sentence alert, and deliver it to a specified recipient via email.

Required tools: `stock`, `email` **Max steps:** 10 **Passing threshold:** $\Phi \geq 0.7$

Grader components ($\sum w_i = 1$):

f_i	Criterion	w_i	Measurement
f_1	Stock price retrieved and numeric	0.40	`isinstance(price, float)`
f_2	Email sent (receipt confirmed)	0.40	SMTP receipt bool
f_3	Ticker symbol present in email body	0.20	substring match

Listing 3: Task 1 Grader

```python
def grade_easy(output: dict) -> float:
    s = 0.0
    if isinstance(output.get("stock_price"), float): s += 0.40
    if output.get("email_sent"):                     s += 0.40
    if "AAPL" in output.get("email_body", ""):       s += 0.20
    return min(s, 1.0)
```

4.3. Task 2 — GDP Growth Analysis (Medium)

`task_medium_gdp_growth`

Goal. Search the web for GDP values of two user-specified countries across two consecutive years. Compute the year-over-year growth rate for each country using the formula below. Deliver a structured email summary including country names, numeric growth rates, and a one-sentence comparative statement.

Required tools: `search`, `calculator`, `email` **Max steps:** 15

$$g = \frac{\mathrm{GDP}_t - \mathrm{GDP}_{t-1}}{\mathrm{GDP}_{t-1}} \times 100\ \%$$

Grader components:

f_i	Criterion	w_i	Measurement
f_1	GDP data retrieved for both countries	0.25	2 numeric values present
f_2	Growth rate computed correctly ($\pm 0.5\%$)	0.30	abs error < 0.5
f_3	Email sent	0.25	receipt bool
f_4	Report contains percentage + comparison	0.20	keyword match

4.4. Task 3 — Multi-Source Structured Report (Hard)

`task_hard_structured_report`

Goal. Build a structured Markdown analytical report by: (1) fetching live stock data and 7-day price history; (2) searching for recent market news about the ticker; (3) computing a 7-day simple moving average (SMA_7); (4) generating a report with *at least* four named sections (Header, Data, Computation, Executive Summary); (5) delivering the full report via email.

Required tools: `stock`, `search`, `calculator`, `email` **Max steps:** 20

$$\text{SMA}_7 = \frac{1}{7}\sum_{i=0}^{6} p_{t-i}$$

Grader components:

f_i	Criterion	w_i	Measurement
f_1	Stock price + 7-day history retrieved	0.15	8 numeric entries
f_2	News snippets fetched	0.15	≥ 1 result returned
f_3	SMA_7 correct (± 0.01)	0.20	abs error threshold
f_4	Report contains ≥ 4 required sections	0.20	section heading presence
f_5	Email sent with full report body	0.20	receipt + body length
f_6	No redundant consecutive tool calls	0.10	log analysis

4.5. Structural Validation

The generated report must include:

- Header
- Data Section
- Computation Section
- Executive Summary

Each section is validated using pattern matching over headings.

5. Reward Function

5.1. Design Rationale

A binary end-of-episode reward is inadequate for meta-agent learning: it provides zero gradient information during the construction phase, which may span 10–20 steps before a single evaluation occurs. OpenCesium uses a **shaped, dense reward** that assigns credit at each step proportional to the marginal contribution of that step toward task completion.

5.2. Reward Decomposition

At each step t, the environment computes:

$$R_t = \alpha\,\Delta C_t \;+\; \beta\,U_t \;+\; \gamma\,E_t \;-\; \delta\,I_t \;-\; \epsilon\,L_t$$

$$\Delta C_t = \Phi_T(A_t, T) - \Phi_T(A_{t-1}, T) \qquad E_t = 1 - \frac{t}{\kappa}$$

Table 2: Reward Component Specifications

Symbol	Name	Semantics	Range	Weight
ΔC_t	Performance gain	Incremental grader score improvement	$[-1, 1]$	$\alpha = 0.40$
U_t	Tool correctness	Required tool used correctly this step	$\{0, 1\}$	$\beta = 0.20$
E_t	Step efficiency	Bonus for acting early in the episode	$[0, 1]$	$\gamma = 0.15$
I_t	Invalid action	Malformed or disallowed action issued	$\{0, 1\}$	$\delta = 0.20$
L_t	Loop penalty	Same action repeated consecutively	$\{0, 1\}$	$\epsilon = 0.10$

5.3. Reward Computation Pipeline

At each step:

1. Apply meta-action a_t
2. Update agent configuration A_t
3. If EVALUATE, compute $\Phi_T(A_t, T)$
4. Compute ΔC_t
5. Compute auxiliary signals:
   - U_t (tool correctness)
   - E_t (efficiency)
   - I_t (invalid actions)
   - L_t (loop repetition)
6. Aggregate into R_t

5.4. Illustrative Step-Level Rewards

Table 3: Example Step-Level Reward Values

Agent Behaviour	R_t
Adds a correctly required tool to configuration	+0.20
EVALUATE called after full correct configuration	+0.40
Adds irrelevant tool not in $\mathcal{U}_T$	−0.05
EVALUATE called on empty configuration	−0.15
Completes task at step 6 out of 10 (40% budget remaining)	+0.06 bonus
Repeats ADDTOOL(`stock`) consecutively	−0.10
Malformed JSON action received	−0.20

5.5. Cumulative Episode Reward

The cumulative return over a full episode is:

$$G = \sum_{t=0}^{\tau} R_t, \qquad G \in [-3.0,\ 2.0] \quad \text{(approximate bounds)}$$

6. Tool Abstraction Layer

6.1. Tool Interface

All tools implement a common abstract base class, ensuring uniform invocation semantics across the registry:

Listing 4: BaseTool Abstract Interface

```python
from abc import ABC, abstractmethod
from pydantic import BaseModel
from typing import Any

class ToolResult(BaseModel):
    success:    bool
    output:     Any
    error:      str   = ""
    latency_ms: float = 0.0

```

```python
class BaseTool(ABC):
    name:        str
    description: str
    input_schema:  dict
    output_schema: dict
    cost: float = 0.0        # Used in reward cost model

    @abstractmethod
    def run(self, params: dict) -> ToolResult: ...
```

6.2. Tool Registry

Table 4: OpenCesium Tool Registry

ID	Backend	Function	Returns	Cost
`stock`	`yfinance`	Live price + OHLCV history; SMA computation	JSON record	0.05
`search`	Tavily / SerpAPI	Ranked web snippets with source attribution	List[dict]	0.10
`calculator`	Python (`math`)	Arithmetic, percentage, moving average	float	0.02
`email`	SMTP / Mailhog	Send formatted message to specified address	receipt dict	0.10

6.3. External API Binding

All tools are backed by real APIs:

- `stock` → yfinance (live financial data)
- `search` → Tavily / SerpAPI (web retrieval)
- `email` → SMTP (Mailhog in development, real SMTP in production)
- `calculator` → deterministic Python evaluation

No tool behavior is mocked during evaluation.

6.4. Cost Model

The per-tool cost $C(u)$ enters the reward function as a budget regulariser:

$$C(u) = \begin{cases} 0.10 & u \in \{\texttt{search}, \texttt{email}\} \\ 0.05 & u = \texttt{stock} \\ 0.02 & u = \texttt{calculator} \end{cases}$$

6.5. Development vs. Production Mode

In **development** mode (CI, automated evaluation), the email tool is backed by **Mailhog** — an in-process SMTP capture server — ensuring no external messages are sent, reproducibility is maintained, and test latency is minimised. In **production** mode, SMTP credentials are injected via environment variables (`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`). The toggle is controlled by `ENV_MODE` in the container environment.

7. Implementation Plan

7.1. Repository Structure

Listing 5: Project Directory Layout

```
opencesium/
```

```
        env/
            __init__.py
            core.py          # OpenCesiumEnv class (reset/step/state)
            models.py        # Pydantic models: Observation, Action, Reward, EnvState
            transitions.py   # Action execution and state transition logic
            episode.py       # Episode tracking, history, termination
        tasks/
            __init__.py
            registry.py      # TASK_REGISTRY dict
            easy.py          # Task 1 definition + grader
            medium.py        # Task 2 definition + grader
            hard.py          # Task 3 definition + grader
        tools/
            __init__.py
            base.py          # BaseTool, ToolResult
            stock.py         # StockTool (yfinance)
            search.py        # SearchTool (Tavily)
            calculator.py    # CalculatorTool
            email_tool.py    # EmailTool (SMTP / Mailhog)
        server/
            app.py           # FastAPI server exposing HTTP endpoints
            routes.py        # /reset, /step, /state, /health routes
        inference.py        # Baseline agent loop (OpenAI client)
        openenv.yaml        # Spec metadata
        Dockerfile
        requirements.txt
        README.md
```

7.2. Development Phases

Table 5: Implementation Schedule

Phase	Module	Deliverables	Priority
P1	`env/models.py`	All Pydantic models; validate with `mypy`	Critical
P2	`env/core.py`	`reset()/step()/state()` skeleton; unit tests	Critical
P3	`tools/`	All 4 tool implementations; mocked test harness	Critical
P4	`tasks/`	3 graders; adversarial test inputs; score assertions	Critical
P5	`env/transitions.py`	Full action dispatch and reward computation	Critical
P6	`server/app.py`	FastAPI HTTP layer; health check endpoint	High
P7	`inference.py`	Agent loop; baseline scores; reproducibility run	High
P8	`Dockerfile`	Container build; docker run verification	High
P9	HF Space deploy	Space creation; secrets config; live smoke test	High
P10	Documentation	`openenv.yaml`, `README.md`, this document	Medium

7.3. Testing Strategy

1. **Unit tests** — Each tool tested in isolation with mocked API responses. Each grader tested with known-correct and known-incorrect outputs, asserting score in $[0, 1]$.
2. **Integration tests** — Full episode runs with scripted action sequences; assert reward trajectory matches expected values.
3. **Compliance validation** — `openenv validate` run in CI on every push; failure blocks merge.
4. **Docker smoke test** — `docker build && docker run` with health check ping to `/reset`.
5. **Reproducibility check** — `inference.py` run three times; assert score standard devia-

tion < 0.01 across runs.

7.4. Strategy Space

The meta-agent can configure execution strategy:

- `react` — iterative reasoning + tool calls
- `plan_execute` — planning followed by execution
- `direct` — minimal reasoning

Strategy affects:

- step efficiency
- tool usage patterns
- final performance

8. Baseline Inference System

8.1. Constraints

Per the hackathon specification, the inference script must: use the OpenAI API client exclusively; read all credentials from environment variables; be named `inference.py` at the project root; complete all three tasks within 20 minutes on a 2-vCPU / 8 GB machine; and produce deterministic, reproducible scores.

8.2. Inference Script

Listing 6: inference.py — Full Baseline Agent Loop

```python
import os, json, time
from openai import OpenAI
from env import OpenCesiumEnv, Action

#           Credentials (injected via environment)

API_BASE_URL = os.environ["API_BASE_URL"]
MODEL_NAME   = os.environ["MODEL_NAME"]
HF_TOKEN     = os.environ["HF_TOKEN"]

client = OpenAI(base_url=API_BASE_URL, api_key=HF_TOKEN)

SYSTEM = """You are an AI workflow engineer.
Given a task description and available tools, issue actions to configure
and run a worker agent. Respond ONLY with valid JSON matching Action schema:
{"action_type": "ADD_TOOL|REMOVE_TOOL|SET_PROMPT|SET_STRATEGY|EVALUATE|NOOP",
 "payload": {<optional key-value params>}}
Do not output any prose, only the JSON object."""

def run_task(task_id: str, max_steps: int = 20) -> dict:
    env  = OpenCesiumEnv(task_id=task_id, max_steps=max_steps)
    obs  = env.reset()
    msgs = [{"role": "system", "content": SYSTEM},
            {"role": "user",   "content": obs.model_dump_json()}]
    total_reward = 0.0
    final_score  = 0.0

    for step in range(max_steps):
        resp = client.chat.completions.create(
            model=MODEL_NAME, messages=msgs,
            temperature=0.0, max_tokens=256, seed=42,
        )
```

```
        raw = resp.choices[0].message.content.strip()
        try:
            action = Action(**json.loads(raw))
        except Exception:
            action = Action(action_type="NOOP", payload={})

        obs, reward, done, info = env.step(action)
        total_reward += reward.total
        final_score   = obs.last_score
        msgs += [{"role": "assistant", "content": raw},
                 {"role": "user",      "content": obs.model_dump_json()}]
        if done:
            break

    return {"task_id": task_id, "steps": step + 1,
            "cumulative_reward": round(total_reward, 4),
            "final_score": round(final_score, 4)}

if __name__ == "__main__":
    tasks = [
        ("task_easy_stock_alert",      10),
        ("task_medium_gdp_growth",     15),
        ("task_hard_structured_report", 20),
    ]
    for tid, ms in tasks:
        t0 = time.time()
        r  = run_task(tid, ms)
        print(f"[{tid}]  score={r['final_score']:.4f}  "
              f"cumR={r['cumulative_reward']:.4f}  "
              f"steps={r['steps']}  time={time.time()-t0:.1f}s")
```

8.3. Expected Baseline Scores

Table 6: Projected Baseline Performance (gpt-4o-mini, temp=0.0)

Task	Difficulty	Grader Score	Cumul. Reward	Avg Steps	Est. Time
task_easy_stock_alert	Easy	0.90–0.95	1.10–1.25	5–8	<30 s
task_medium_gdp_growth	Medium	0.70–0.82	0.80–1.00	9–13	<90 s
task_hard_structured_report	Hard	0.48–0.62	0.55–0.75	16–20	<180 s
Total runtime					<6 min

9. Deployment Architecture

9.1. Dockerfile

Listing 7: Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential curl && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 7860
ENV PYTHONUNBUFFERED=1
ENV ENV_MODE=development
```

```

HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD curl -f http://localhost:7860/health || exit 1

CMD ["uvicorn", "server.app:app", "--host", "0.0.0.0", "--port", "7860"]
```

9.2. openenv.yaml

Listing 8: openenv.yaml

```yaml
id:          opencesium-v1
name:        OpenCesium
version:     "1.0.0"
description: >
  Meta-agent workflow automation environment. An outer LLM agent
  constructs and configures tool-using sub-agents to solve structured
  business workflows involving financial data, computation, and delivery.
author:      Pardhu
tags:        [openenv, meta-agent, workflow-automation, finance, tool-use]

observation_space:
  type: object
  fields:
    task_id: str; task_description: str; available_tools: list[str]
    agent_config: dict; last_score: float; last_reward: float
    step_index: int; max_steps: int; history: list; done: bool

action_space:
  type: object
  enum: [ADD_TOOL, REMOVE_TOOL, SET_PROMPT, SET_STRATEGY, EVALUATE, NOOP]
  payload: dict (tool, prompt, strategy, params)

tasks:
  - {id: task_easy_stock_alert,       difficulty: easy,   max_steps: 10}
  - {id: task_medium_gdp_growth,      difficulty: medium, max_steps: 15}
  - {id: task_hard_structured_report, difficulty: hard,   max_steps: 20}

reward: {type: dense, range: [-0.5, 2.0], partial_progress: true}
runtime: {python: "3.11", memory_gb: 8, vcpu: 2}
```

10. Evaluation and Judging Alignment

10.1. Judging Criteria Mapping

Table 7: Hackathon Scoring Criteria vs. OpenCesium Design Decisions

Criterion	Weight	OpenCesium Response
Real-world utility	30%	Workflow automation is a high-value, daily enterprise task. All three scenarios (stock alert, GDP analysis, structured reporting) mirror real pipelines operated by finance and analytics teams. The environment fills a genuine gap — no existing OpenEnv submission benchmarks meta-agent construction for business workflows.
Task & grader quality	25%	Three tasks with clear difficulty progression and four to six independently measurable sub-criteria each. All graders are deterministic (no randomness, no judge model), produce scores in $[0, 1]$, and are tested with adversarial inputs. The hard task is expected to challenge frontier-class models (projected score 0.48–0.62 for GPT-4o-mini).
Environment design	20%	Clean episode state via Pydantic models; sensible action and observation spaces; trajectory-dense reward with decomposed components; clear termination conditions; zero state leakage between episodes.
Code quality & spec	15%	Full OpenEnv v1.0 compliance, `openenv validate` in CI, typed models throughout, working Dockerfile verified locally, HF Space deployment with health check, reproducible inference script.
Creativity & novelty	10%	Meta-agent framing (agent configuring agents) is novel in the OpenEnv ecosystem. The reward design — combining performance gain, tool correctness, efficiency, and loop penalties — is an original composition. The SetStrategy action (ReAct vs. Plan-Execute vs. Direct) provides an additional dimension not seen in existing environments.

10.2. Pre-Submission Checklist

Table 8: Automated Validation Gate Requirements

Check	Target Status
HF Space deploys; `/health` returns HTTP 200	✓ Required
`reset()` returns valid `Observation`	✓ Required
`openenv validate` passes with zero errors	✓ Required
`docker build && docker run` succeeds	✓ Required
`inference.py` completes without exception	✓ Required
Baseline scores reproduced (std < 0.01) $\times 3$	✓ Required
3 tasks enumerated; all grader scores in $[0, 1]$	✓ Required
`API_BASE_URL`, `MODEL_NAME`, `HF_TOKEN` defined	✓ Required
Total inference runtime < 20 min on 2 vCPU / 8 GB	✓ Required
Graders return distinct scores (not constant)	✓ Required

10.3. Failure Mode Mitigations

Table 9: Disqualification Risks and Mitigations

Risk	Mitigation Strategy
Environment does not deploy	Dockerfile tested locally; HF Space health-checked before submission. `/health` endpoint added explicitly.
Grader always returns same score	Graders are multi-component pure functions; tested with both correct and incorrect outputs; score assertions in CI.
No baseline inference script	`inference.py` in project root; named explicitly; runs end-to-end before submission.
Plagiarism flag	Entirely original environment design; meta-agent framing not present in any known OpenEnv submission.
Timeout on evaluation machine	All tool calls time-boxed; total projected runtime < 6 min; Mailhog used in CI (no network latency).

10.4. Novel Contributions Summary

OpenCesium makes four original contributions to the OpenEnv ecosystem:

1. **Meta-agent evaluation framework** — Benchmarks the construction of agents, not merely their execution, shifting the evaluation target from policy output to policy design.
2. **Agent configuration as structured action space** — Defines a typed, semantically rich action vocabulary (AddTool, SetPrompt, SetStrategy, Evaluate) that goes beyond simple tool invocation.
3. **Trajectory-dense reward over construction** — Provides meaningful learning signal at every configuration step, not only at the terminal evaluation call.
4. **Composable grader architecture** — Each grader is a weighted sum of independently testable sub-criteria, enabling fine-grained diagnostics of agent failure modes.

Conclusion

OpenCesium formalises a problem class that existing agent benchmarks overlook: the automated construction and optimisation of tool-using workflow agents. By grounding the environment in genuine business automation scenarios — financial alerting, economic analysis, and structured report generation — it achieves the hackathon's mandate for real-world utility while simultaneously introducing a novel and technically substantive evaluation axis.

The full system satisfies every mandatory OpenEnv v1.0 requirement: Pydantic-typed observation, action, and reward models; clean `reset()/step()/state()` semantics; three graded tasks spanning easy to hard; a shaped, trajectory-dense reward function with partial progress signals; a reproducible inference script using the OpenAI client; and a containerised deployment to Hugging Face Spaces. The implementation plan and repository structure laid out in this document provide a direct path from specification to a passing submission.

References

[1] Zhou S. et al. (2023). WebArena: A realistic web environment for building autonomous agents. *ICLR 2024*.

[2] Qin Y. et al. (2023). ToolLLM: Facilitating large language models to master 16000+ real-world

APIs. *arXiv:2307.16789*.

[3] Liu X. et al. (2023). AgentBench: Evaluating LLMs as agents. *arXiv:2308.03688*.

[4] Significant Gravitas (2023). AutoGPT: An autonomous GPT-4 experiment. https://github.com/Significant-Gravitas/AutoGPT.

[5] Chase H. (2022). LangChain: Building applications with LLMs through composability. https://github.com/langchain-ai/langchain.

[6] Shinn N. et al. (2023). Reflexion: Language agents with verbal reinforcement learning. *NeurIPS 2023*.

[7] Colvin S. et al. (2023). Pydantic v2. https://docs.pydantic.dev.

[8] Aroussi R. (2019). yfinance: Yahoo Finance market data downloader. https://pypi.org/project/yfinance.

[9] Tavily AI (2023). Tavily Search API. https://tavily.com.